UNIVERSIDAD CATOLICA DEL MAULE

Computación de Alto Rendimiento

Javier E. Rojas Núñez

jerojas@ucm.cl

Contenidos

UNIVERSIDAD CATOLICA DEL MAULE

Resultado de aprendizaje: Aplicar la programación multi-hilo para el análisis de una gran cantidad de datos a través de sistemas que tengan una gran capacidad de cómputo.

- **Computación paralela**
  - La computación paralela y sus usos
  - Multiprocess en python
  - Memoria compartida y sincronización
  - Más herramientas de sinconización

Evaluaciones (40 %)

- Ejercios o tareas (16%)
- Taller de cierre (24%)

Bibliografía

UNIVERSIDAD CATOLICA DEL MAULE

Principal

- Roman Trobec, Boštjan Slivnik, Patricio Bulić, Borut Robič (2018). Introduction to Parallel Computing: From Algorithms to Programming on State-of-the-Art Platforms. Editorial: Springer.
- Kirk, D. B., & Wen-Mei, W. H. (2016). Programming massively parallel processors: a hands- on approach, Third Edition. Morgan Kaufmann, Elsevier.
- Timothy G. Mattson, Yun (Helen) He, Alice E. Koniges (2019). The OpenMP Common Core: Making OpenMP Simple Again (Scientific and Engineering Computation). The MIT Press.
- José Enrique Roman Moltó, José Miguel Alonso Ábalos, Ignacio Blanquer Espert, David Guerrero López, Jacinto Javier Ibañez Gonzalez, Enrique Ramos Peinado, Fernando Alvarruiz Bermejo (2018). Ejercicios de programación paralela con OpenMP y MPI. Editorial Universitat Politècnica de València; Edición Nº1, 2018. Georg Hager (2017). Introduction to High Performance Computing for Scientists and Engineers. Editorial: CRC Press; 1st Edition.

Complementaria

- Georg Hager (2017). Introduction to High Performance Computing for Scientists and Engineers. Editorial: CRC Press; 1st Edition.
- Ruud Van Der Pas, Eric Stotzer, Christian Terboven (2017). Using OpenMP-The Next Step: Affinity, Accelerators, Tasking, and SIMD (Scientific and Engineering Computation). The MIT Press; 1st edition.

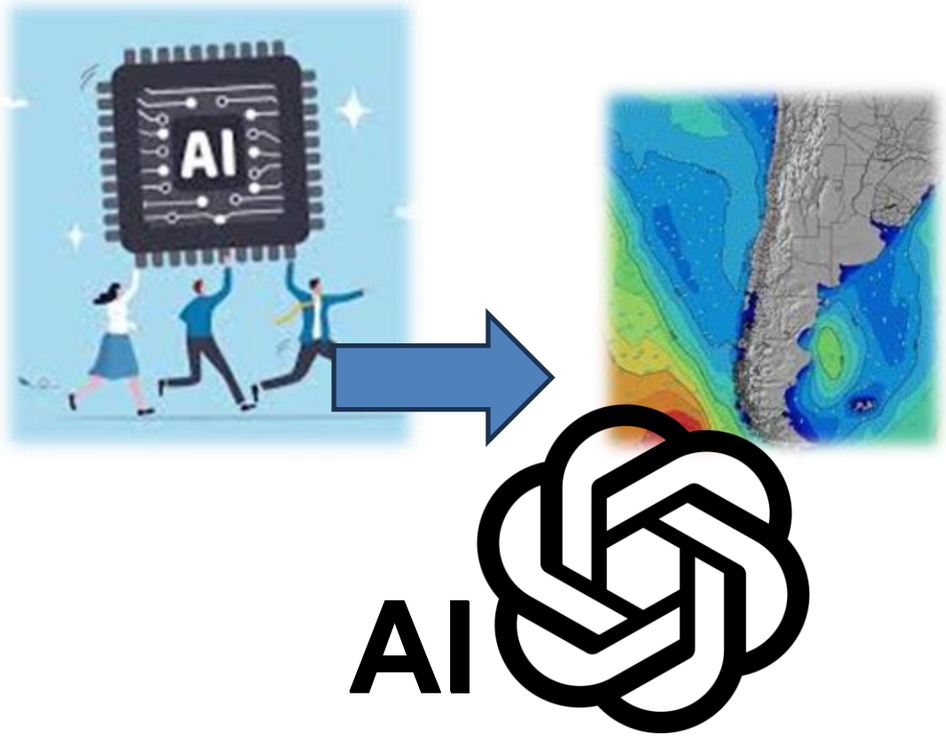
UNIVERSIDAD CATOLICA DEL MAULE

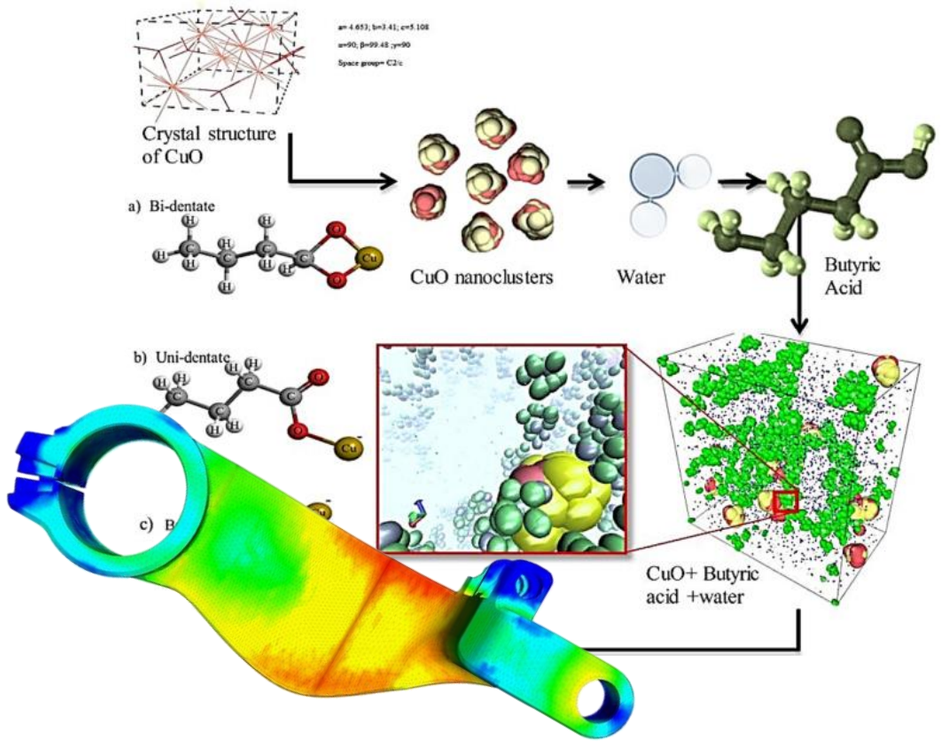


Usos y tipos de Computación paralela

Ejemplos de uso

El uso de estas tecnologías se desarrollan en diversas áreas, las cuales tienen un impacto en diversos ámbitos

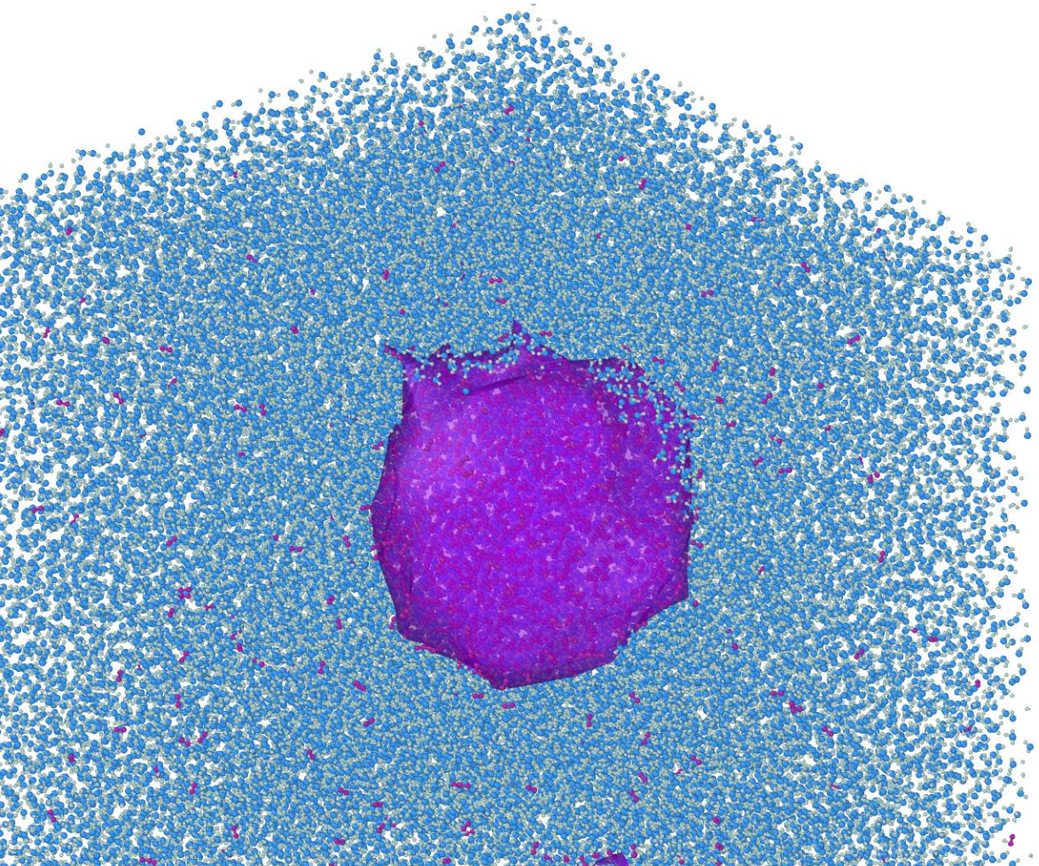


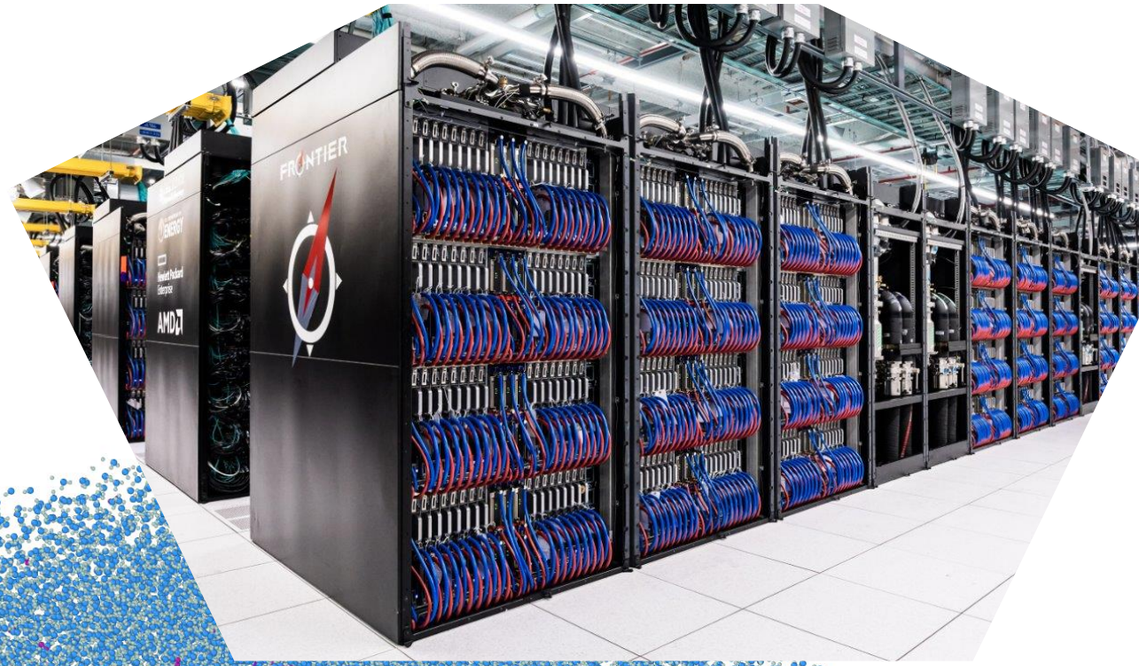


Simulaciones

Mi experiencia personal

La computación paralela hace posible mi investigación en Ciencia de Materiales. Donde la computación paralela permite **reducir los tiempos de cómputo** para simulaciones en **Dinámica Molecular**





En búsqueda de más...

Las computadoras tienen la capacidad de procesar información a gran velocidad, y su pontencia está asociada a la frecuencia de rejoj.

- La frecuencia ha alcanzado su limite
- Existen tareas que todavía le toman mucho tiempo a los equipos de computo actuales
- La potencia de cómputo se consigue mediante sistemas multihilo o multinodo



Tareas segregadas

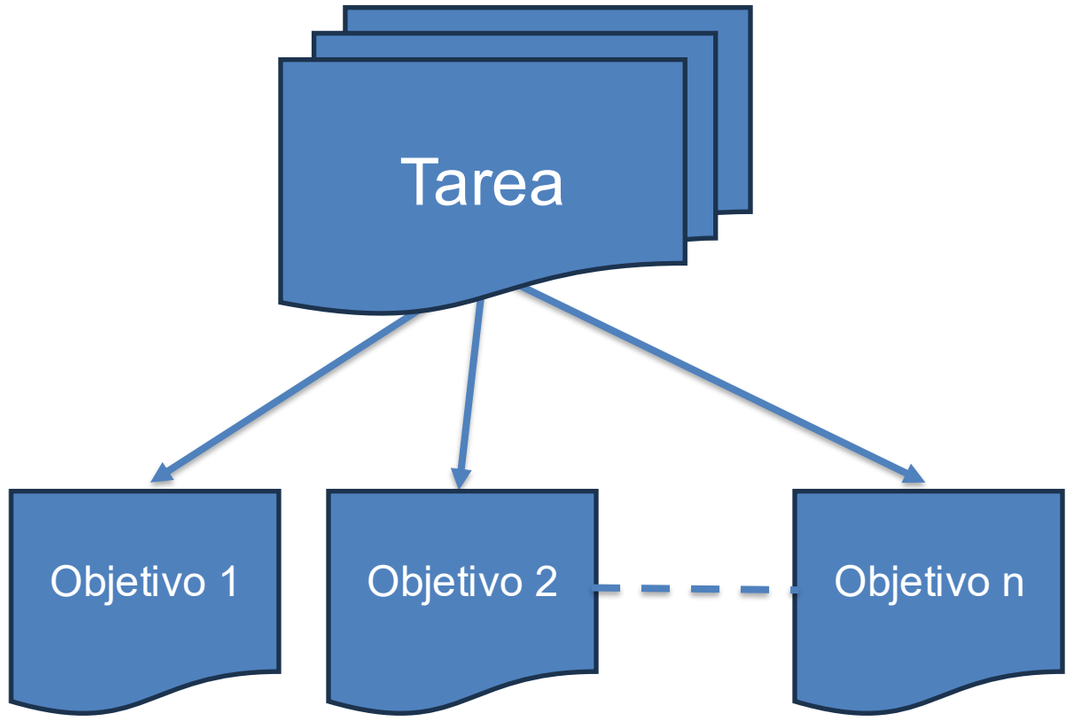
Hay tareas (o cálculos) que pueden repartirse entre varias subtareas individuales

- Una suma de muchos elementos

$$\sum_{n=1}^{1e6} n$$

- Puede ser repartida para reducir el tiempo total

$$\sum_{n=1}^{5\times10^5} n + \sum_{n=5\times10^5+1}^{10^6} n$$



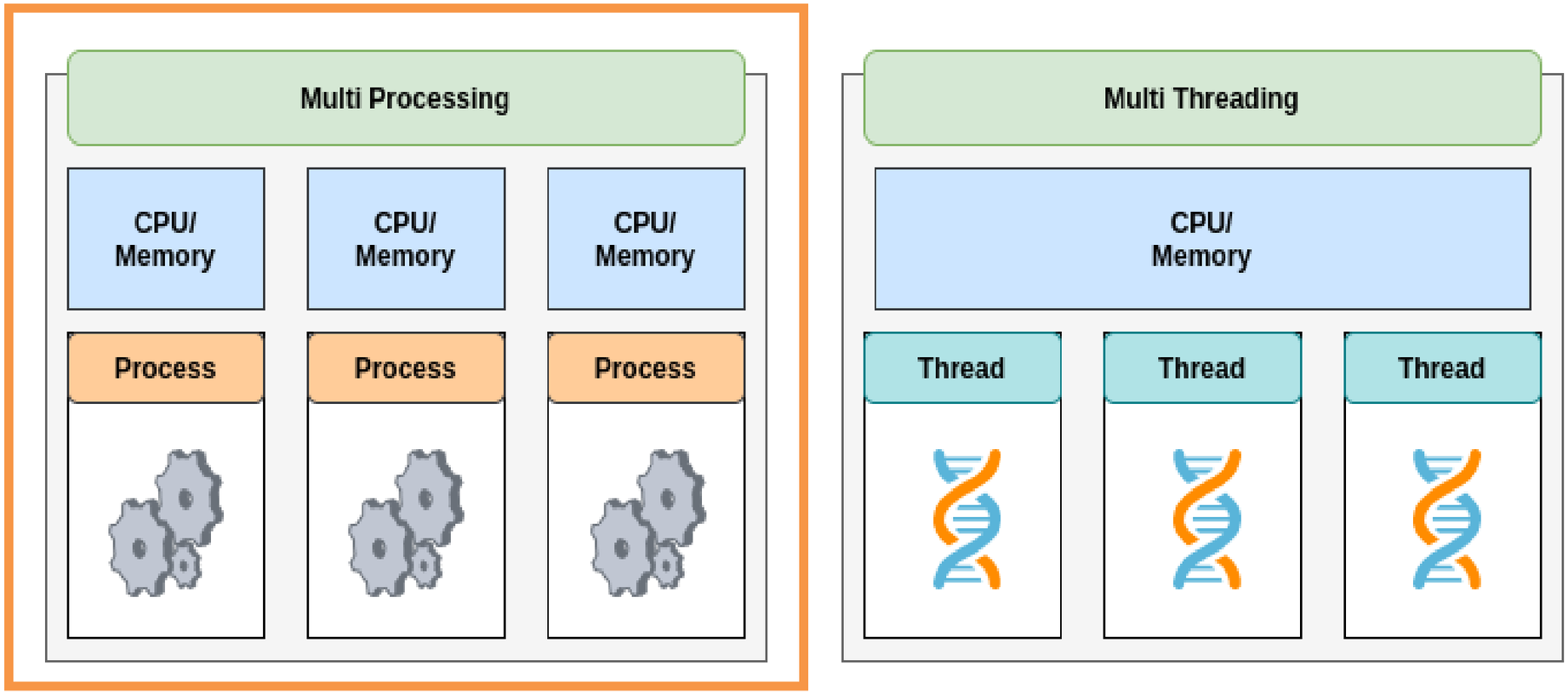
UNIVERSIDAD CATOLICA DEL MAULE



Computación paralela

Computación peralela

Para trabajar en las subtareas, es necesario utilizar técnicas de procesamiento multihilo o multiproceso.



Nos cetraremos en las técnicas multiproceso dentro de un mismo equipo



Introducción a multiprocesing

UNIVERSIDAD CATOLICA DEL MAULE

Multiprocessing y python

Mutiprocessing es una librería de python que permite la ejecución de un programa en distintos sub-procesos, haciendo efectivo un procesamiento paralelo

Elementos necesarios:

- Librería de multiprocesing
- Una rutina (función) para realizar la tarea
- Una rutina que crea y coordina las distintas tareas

```
import multiprocessing as mp
from time import sleep

def fun_paralela(iid, arreglo):
    arreglo[iid] = iid
    sleep(5)
    print('Hola Mundo – No {:02d} Completado!'.format(iid),
          flush=True)


if __name__ == '__main__':
    p_num = 8
    arr = mp.Array('i', [0 for i in range(p_num)],
                   lock=False)
    # Creando lista de procesos
    procs = [mp.Process(target=fun_paralela, args=(i, arr))
             for i in range(p_num)]
```

Procesos de Multiprocessing

UNIVERSIDAD CATOLICA DEL MAULE

Para poder comenzar a paralelizar requerimos de crear lo objetos que contendrán las rutinas que vamos a correr en paralelo. Para esto necesitamos usar la clase Process de la librería Multiprocesing:

- p = Process(target=funcion, name='mi_proceso', arg=(1,'Soy texto'): Clase que crea los objetos de procesos (enviado a p)
- p.start(timeout=None): Inica una tarea y sigue el código
- p.join(): Espera que la tarea termine y sigue con el código

```
if __name__ == '__main__':
    p_num = 8
    arr = mp.Array('i', [0 for i in range(p_num)],
                   lock=False)
    # Creando lista de procesos
    procs = [mp.Process(target=fun_paralela, args=(i, arr))
             for i in range(p_num)]

    # Corriendo tareas
    for p in procs:
        p.start()

    # Esperar a que terminen las tareas
    for p in procs:
        p.join()

    print(('\nShow all ids:\n' + p_num*' {:d}').format(*arr))
```

Mi primer programa paralelo

UNIVERSIDAD CATOLICA DEL MAULE

Usando estos conceptos cree su primer programa paralelo, este programa simplemente deberá imprimir en pantalla la frase "Hola Mundo – No *N* Completado!" Donde *N* sea el número del proceso que ejecutó la tarea.

- Estructure el programa con su función paralela para enviar mensajes
- En la rutina principal cree los objetos de ejecución e inicialícelos todos en secuencia usando un for loop
- Espere la finalización con otro for loop con "p.join()" para el correcto cierre del programa

```
def fun_paralela(iid):
 print('Hola Mundo – No {:02d} Completado!'.format(iid),
 flush=True)

if __name__ == '__main__':
    p_num = 8
    arr = mp.Array('i', [0 for i in range(p_num)],
                   lock=False)
    # Creando lista de procesos
    procs = ???

    # Corriendo tareas
    for p in procs:
        p.start()

    # Esperar a que terminen las tareas
    for p in procs:
        p.join()
```

Elementos compartidos de memoria

UNIVERSIDAD CATOLICA DEL MAULE

Usando las funciones Array y Value de Multiprocessing es posible crear elementos de memoria compartida. Su uso es desaconsejado por no ser un procedimiento seguro.

- Array(type, ini, lock): Crea un array de memoria compartida. Type define el tipo de variable que contiene el array, mientras que ini es un elemento inicializador.
- Value(type, lock): Crea un valor de memoria compartida para su uso en multiples procesos

Símbolo	Tipo	Tamaño en Bytes
'b'	signed char	1
'B'	unsigned char	1
'h'	signed short	2
'H'	unsigned short	2
'i'	signed int	4
'I'	unsigned int	4
'l'	signed long	8
'L'	unsigned long	8
'q'	signed long long	8
'Q'	unsigned long long	8
'f'	float	4
'd'	double	8



Actividad

Sumando Números

Visto los elementos de procesamiento paralelo. Cree una rutina que sume los números consecutivos desde el 1 al 10.000.000, usando los elementos vistos en esta clase. Reparta la carga entre diferentes cantidades de procesos y pruebe el tiempo que se demoran

- Crear la rutina que sume los numeros consecutivos mediante un programa en paralelo
- Probar distintas tamaños de sumas para probar la robustes y velocidad de la rutina
- Utilizar distintos formatos de información para almacenar y procesar los resultados

```
def fun_paralela(iid):
  ...

if __name__ == '__main__':
    p_num = 8
    arr = mp.Array('i', [0 for i in range(p_num)],
                   lock=False)
    # Creando lista de procesos
    procs = ???

    # Corriendo tareas
    for p in procs:
        p.start()

    # Esperar a que terminen las tareas
    for p in procs:
        p.join()
```